

DNS Analysis

September 30, 2025

1 Project: DNS Analysis

ITIS 4260 Efren Antonio

```
[1]: # import pandas and sys libraries
import pandas as pd
import sys
```

```
[2]: # insert the path to the 'freq-master' directory into the system path
sys.path.insert(1, "./freq-master")

# import the FreqCounter class from the freq module
from freq import FreqCounter

# create an instance of the FreqCounter class and load the frequency table from
↳ the file
freq_counter = FreqCounter()
freq_counter.load('./freq-master/freqtable2018.freq')
```

```
[3]: # load the DNS log JSON file into a DataFrame
dns_df = pd.read_json('dnslog.json', lines=True)
dns_df.head()
```

```
[3]: Unnamed: 0          ts id.orig_h id.orig_p id.resp_h \
0      16583  2024-04-07 12:00:00 10.0.1.11      55555 10.0.0.10
1      16585  2024-04-07 12:00:00 10.0.1.11      47742 10.0.0.10
2      16586  2024-04-07 12:00:00 10.0.1.11      39923 10.0.0.10
3      16587  2024-04-07 12:00:00 10.0.1.11      57318 10.0.0.10
4      16584  2024-04-07 12:00:00 10.0.1.11      54568 10.0.0.10

      id.resp_p proto          query qtype \
0           53  udp  9.a.6.e.f.9.c.c.a.8.5.e.5.b.5.8.0.0.0.0.0.0...      12
1           53  udp                                1.0.16.172.in-addr.arpa      12
2           53  udp  2.0.0.0.1.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0...      12
3           53  udp                                9.0.0.10.in-addr.arpa      12
4           53  udp                                2.0.16.172.in-addr.arpa      12

      qtype_name rcode      answers
```

```

0      PTR      3      -
1      PTR      3      -
2      PTR      3      -
3      PTR      0  dc02.test.int
4      PTR      0  ts01.test.int

```

```

[4]: # load the top-1m.csv file into a DataFrame
alex_a_df = pd.read_csv('top-1m.csv', header=None, names=['rank', 'domain'])
alex_a_domains = set(alex_a_df['domain'])
alex_a_df.head()

```

```

[4]:      rank      domain
0      1      google.com
1      2  microsoft.com
2      3  www.google.com
3      4  facebook.com
4      5  doubleclick.net

```

```

[8]: # Function to extract domain names from DNS queries
def extract_domain_names(query_list):
    domain_names = []
    for query in query_list:
        try:
            parts = query.split(".")
            if len(parts) > 1:
                domain_names.append(parts[-2]) # Extract second-to-last part
            ↪as domain name
        else:
            domain_names.append(None) # Handle single-word queries
    except AttributeError:
        domain_names.append(None) # Handle non-string entries
    return domain_names

# Extract domain names from the 'query' column
dns_df['domain'] = extract_domain_names(dns_df['query'])

# Remove rows with invalid domain names
dns_df.dropna(subset=['domain'], inplace=True)

```

```

[11]: # Function to calculate frequency for each domain
def calculate_frequency(domain_names):
    unique_domains = set(domain_names)
    domain_freq = {dname: freq_counter.probability(dname)[0] for dname in
    ↪unique_domains}
    return [domain_freq[dname] for dname in domain_names]

# Function to check Alexa Top 1M membership

```

```
def check_alexamembership(domain_names):
    return [domain in alexa_domains for domain in domain_names]

# Calculate frequency scores
dns_df['frequency'] = calculate_frequency(dns_df['domain'])
dns_df['in_alexam1'] = check_alexamembership(dns_df['domain'])

# Display the enriched DataFrame
dns_df.head()
```

```
[11]: Unnamed: 0          ts id.orig_h id.orig_p id.resp_h \
0      16583  2024-04-07 12:00:00 10.0.1.11      55555 10.0.0.10
1      16585  2024-04-07 12:00:00 10.0.1.11      47742 10.0.0.10
2      16586  2024-04-07 12:00:00 10.0.1.11      39923 10.0.0.10
3      16587  2024-04-07 12:00:00 10.0.1.11      57318 10.0.0.10
4      16584  2024-04-07 12:00:00 10.0.1.11      54568 10.0.0.10

      id.resp_p proto          query qtype \
0          53  udp  9.a.6.e.f.9.c.c.a.8.5.e.5.b.5.8.0.0.0.0.0.0...      12
1          53  udp                                1.0.16.172.in-addr.arpa      12
2          53  udp  2.0.0.0.1.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0...      12
3          53  udp                                9.0.0.10.in-addr.arpa      12
4          53  udp                                2.0.16.172.in-addr.arpa      12

      qtype_name rcode      answers      domain frequency in_alexam1
0          PTR      3          -      ip6      0.3383      False
1          PTR      3          -  in-addr      6.8947      False
2          PTR      3          -      ip6      0.3383      False
3          PTR      0  dc02.test.int  in-addr      6.8947      False
4          PTR      0  ts01.test.int  in-addr      6.8947      False
```

```
[13]: def mapped_ip(ip, dns_df):

    # Filter the DNS log for entries where the resolved IP matches the given IP
    matched_entries = dns_df[dns_df['answers'] == ip]

    if matched_entries.empty:
        return []

    # Count occurrences of each domain
    domain_counts = matched_entries['domain'].value_counts().items()

    return list(domain_counts)

# Example usage with test IPs from problem 2
test_ips = ["99.84.181.52", "137.245.120.50", "104.244.42.1", "172.20.56.90"]
```

```
for ip in test_ips:
    print(f"Domains resolving to {ip}:", mapped_ip(ip, dns_df))
```

```
Domains resolving to 99.84.181.52: []
Domains resolving to 137.245.120.50: []
Domains resolving to 104.244.42.1: [('twitter', 1583)]
Domains resolving to 172.20.56.90: []
```

```
[71]: # Identify suspicious domains (ending in .net, not in Alexa 1M, and low
      ↪ frequency score)
suspicious_domains = dns_df[dns_df['query'].str.contains(r'\.net(?:,|$)',
      ↪ regex=True, na=False) & (~dns_df['in_alex_1m']) & (dns_df['frequency'] < 4)]

# Display suspicious domains
print("Suspicious Domains:")
print(suspicious_domains[['query', 'frequency', 'in_alex_1m']])
```

Suspicious Domains:

	query	frequency	in_alex_1m
1311	RT4x78.net	1.8976	False
1490	RT4x78.net	1.8976	False
1860	www.RT4x78.net	1.8976	False
1912	www.RT4x78.net	1.8976	False
4182	www.RT4x78.net	1.8976	False
4345	www.RT4x78.net	1.8976	False
5665	www.RT4x78.net	1.8976	False
7196	www.RT4x78.net	1.8976	False
10595	www.RT4x78.net	1.8976	False
14312	www.RT4x78.net	1.8976	False
15714	www.RT4x78.net	1.8976	False
23279	www.RT4x78.net	1.8976	False
26102	RT4x78.net	1.8976	False